Threat Modeling & Quantitative Risk Assessment

COMSATS University Islamabad | CYC386 Secure Software Design and Development
Team: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)

Threat Modeling & Quantitative Risk Assessment (CDF Unit 3)

Course: CYC386 – Secure Software Design and Development

Exam: Midterm Lab Exam — 1-Week DevSecOps Security Sprint (Fall 2026)

Institution: COMSATS University Islamabad, Department of Computer Science

Instructor: Engr. Muhammad Ahmad Nawaz

Project: SecureDrop – Enterprise Document Vault Platform (`SecureApp-Sprint-Team1`)

Team Members

- **Talal Nadeem Awan** (FA23-BCT-051)
- **Shahroze Afaq** (FA23-BCT-039)

---

1. Introduction & Methodology

Threat modeling identifies, enumerates, and prioritizes potential threats systematically before deployment. This report implements:

1. **Data Flow Diagrams (DFDs):** Context Diagram (Level-0) and Decomposed Level-1 DFD.

2. **STRIDE Threat Classification:** Systematic evaluation of Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege across each DFD element.

3. **Attack Trees:** Graphical and structural decomposition of two critical threat paths: OS Command Injection and Native Buffer Overflow.

4. **CVSS v3.1 Quantitative Risk Scoring:** Standardized scoring with full vector strings and technical justifications.

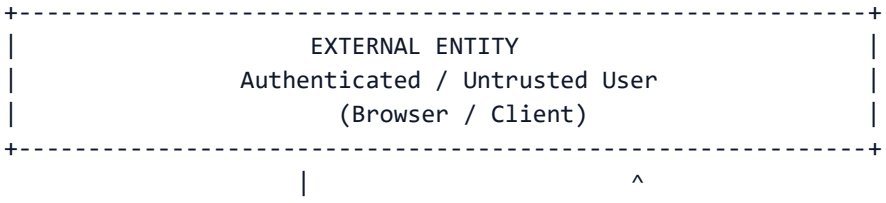
5. **Prioritized Risk Register:** Linked to Jira sprint backlog items (`SASS`).

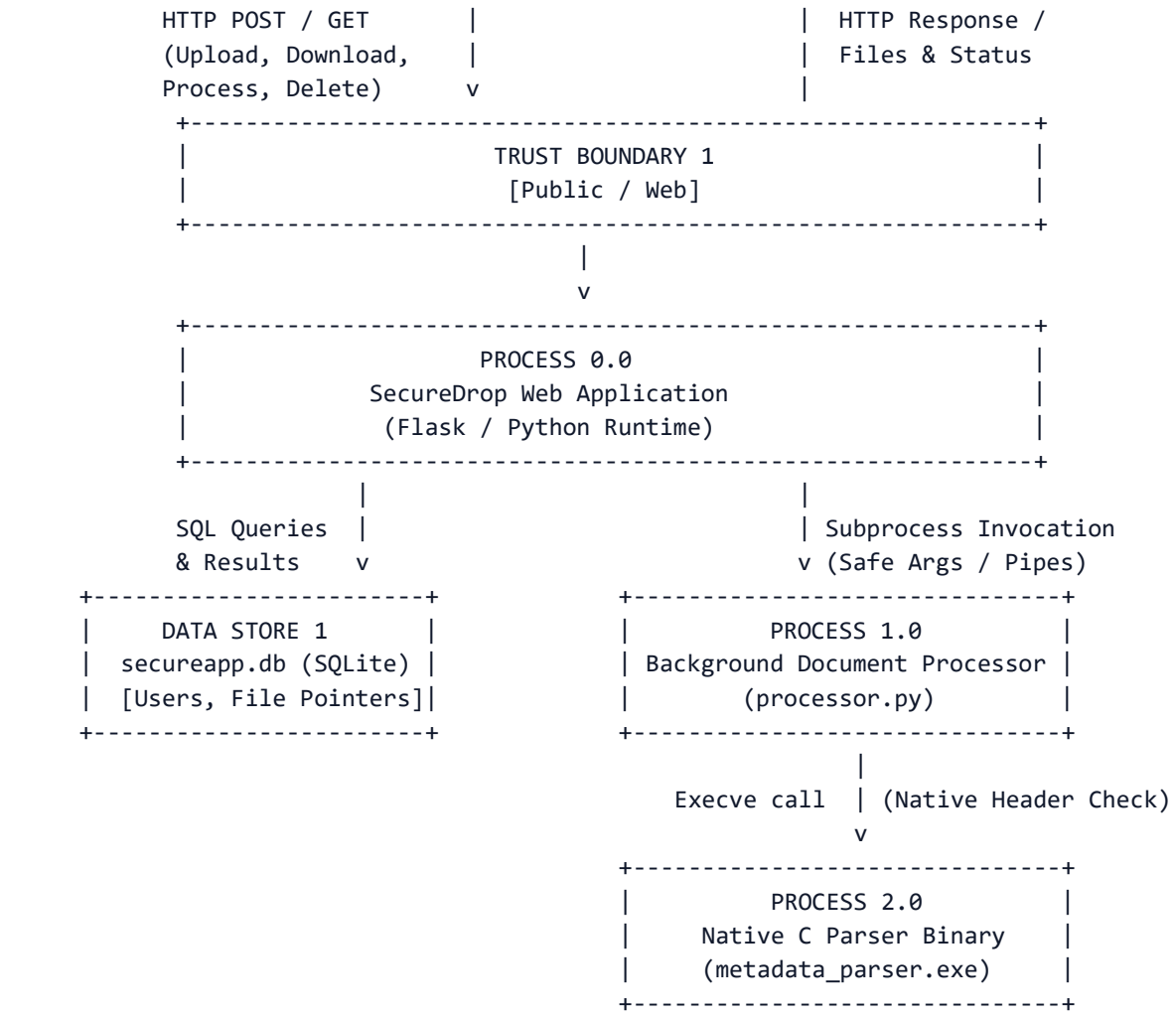
6. **Residual Risk & Mitigation Analysis.**

---

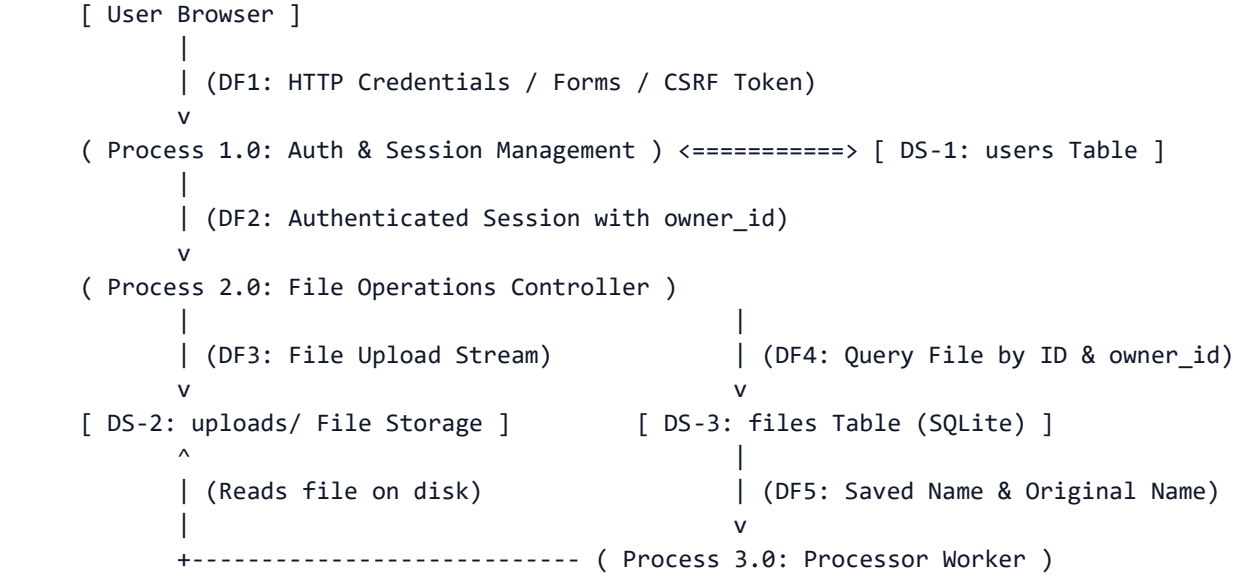
2. Data Flow Diagrams (DFDs)

2.1 Context Diagram (Level-0 DFD)





2.2 Decomposed Level-1 Data Flow Diagram (DFD)



```
                    |
                    | (DF6: Passes metadata string)
                    v
          ( Process 4.0: Native C Parser )
                    |
                    | (Validates bounds & stack canary)
                    v
             [ OS Exit Code ]
```

---

3. Systematic STRIDE Threat Analysis

Each element of the Level-1 DFD is subjected to STRIDE analysis:

| Element ID | Element Name | DFD Type | STRIDE Category | Threat Description | Mitigating Security Control |

| :--- | :--- | :--- | :--- | :--- | :--- |

| **P-1.0** | Authentication Controller | Process | **Spoofing** | Attacker spoofs identity by guessing weak passwords or hijacking session cookies. | Enforced password length (>=8), salted hashing via PBKDF2/scrypt, `HttpOnly` and `SameSite=Strict` cookies. |

| **P-2.0** | File Operations Controller | Process | **Tampering** | Attacker deletes files belonging to another user via forged cross-origin requests. | Anti-CSRF Synchronizer Token enforced on all state-changing routes (`validate_csrf()`). |

| **P-2.0** | File Operations Controller | Process | **Elevation of Privilege** | Attacker accesses unauthorized files by altering URL `file_id` parameter (IDOR). | Object-level authorization strictly checks `WHERE id = ? AND owner_id = session['user_id']`. |

| **P-3.0** | Processor Worker | Process | **Elevation of Privilege** | Attacker injects shell metacharacters (`;`, `&`, `\|`) to gain Remote Code Execution (RCE). | Parameterized `subprocess.run(..., shell=False)` with argument arrays and strict regex validation. |

| **P-4.0** | Native C Metadata Parser | Process | **Denial of Service / EoP** | Oversized metadata string overflows fixed stack buffer, crashing app or hijacking control flow. | Explicit bounds checking (`len < sizeof(buffer)`), bounded `memcpy`, and compiler stack canary (`-fstack-protector-strong`). |

| **DF-1** | User Request Channel | Data Flow | **Tampering** | Man-in-the-Middle attacker intercepts or alters HTTP requests. | Production requirement: HTTPS enforcement, `Strict-Transport-Security` header. |

| **DF-2** | UI Presentation Layer | Data Flow | **Information Disclosure** | Sensitive application framed inside malicious transparent iframe (Clickjacking). | Response headers: `X-Frame-Options: DENY` and CSP `frame-ancestors 'none'`. |

| **DS-1** | `users` SQLite Database | Data Store | **Information Disclosure** | Attacker extracts database file to read plaintext passwords. | Salted hashes generated via `generate_password_hash()`, restricted file permissions on `secureapp.db`. |

| **DS-2** | `uploads/` Storage Directory | Data Store | **Tampering** | Attacker overwrites existing files via path traversal or predictable filenames. | UUID-style random storage filenames (`secrets.token_hex(16) + '.txt'`), path traversal sanitization. |

---

4. Attack Trees for Critical Threats

4.1 Attack Tree 1: Remote Code Execution (RCE) via OS Command Injection

```
Goal: Execute Arbitrary Code on Host Server (RCE)
├── OR 1: Exploit Command Concatenation in File Processing (CWE-78)
│   ├── 1.1: Identify processing endpoint accepting user input (/process)
│   ├── 1.2: Craft filename payload with shell operators: "report.txt & echo
LAB_MARKER"
│   └── 1.3: Verify shell interpretation (shell=True executes injected command)
│       └── [REMEDIATION]: Use execve-style argument list with shell=False & regex
allowlist
│
└── OR 2: Path Traversal to Execute Injected Script
    ├── 2.1: Upload malicious script disguised as text: "../../bin/malicious.sh"
    └── 2.2: Force server-side execution
        └── [REMEDIATION]: Random hex token saved names + strict [A-Za-z0-9_-
]{1,64}\.txt pattern
```

4.2 Attack Tree 2: Memory Corruption / Privilege Escalation via Buffer Overflow

```
Goal: Crash Host Process or Hijack Control Flow (CWE-120 / CWE-121)
├── OR 1: Overflow Stack Buffer in Native C Parser
│   ├── 1.1: Analyze native component metadata.c accepting external input
│   ├── 1.2: Identify unbounded copy loop: for(size_t i=0; i<=len; i++) filename[i]
= input[i]
│   ├── 1.3: Supply oversized input (63 bytes into 32-byte fixed stack buffer)
│   ├── 1.4: Overwrite stack canary, saved frame pointer, and return address
│   └── 1.5: Trigger process termination or redirection to shellcode
│       └── [REMEDIATION]: Strict bounds checking (len < sizeof(filename)), bounded
memcpy,
│                          and compiler mitigation (-fstack-protector-strong / ASLR
/ DEP).
```

---

5. CVSS v3.1 Quantitative Risk Assessment

Every identified vulnerability is evaluated using the FIRST Common Vulnerability Scoring System (CVSS v3.1):

| Risk ID | Vulnerability Class | CWE | CVSS v3.1 Vector String | Base Score | Severity | Justification |

| :--- | :--- | :--- | :--- | :---: | :---: | :--- |

| **R-01** | OS Command Injection | CWE-78 | `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` | **8.8** | **High** | Authenticated network attacker can pass shell metacharacters resulting in total compromise of confidentiality, integrity, and availability. |

| **R-02** | Insecure Direct Object Reference (IDOR) | CWE-639 | `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N` | **8.1** | **High** | Authenticated attacker can view, download, or delete any user's document simply by enumerating sequential integer IDs. |

| **R-03** | Stack-based Buffer Overflow | CWE-121 | `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H` | **5.5** | **Medium** | Local/authenticated component receives oversized input causing stack smashing and process termination (DoS). If exploited for control flow: Score reaches 7.8 (High). |

| **R-04** | Cross-Site Request Forgery (CSRF) | CWE-352 | `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N` | **6.5** | **Medium** | Unauthenticated remote attacker tricks logged-in user into triggering state-changing document deletion via malicious form without user consent. |

| **R-05** | Clickjacking / UI Redressing | CWE-1021 | `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N` | **4.3** | **Medium** | Attacker frames legitimate application inside transparent iframe to trick user into clicking unintended buttons. |

---

6. Prioritized Risk Register & Jira Traceability

| Risk ID | Priority | Vulnerability | Remediation Strategy | Jira Epic | Jira Issue Key |

| :--- | :---: | :--- | :--- | :--- | :--- |

| **R-01** | **P1 (Critical)** | OS Command Injection | Parameterized `subprocess.run(shell=False)` + Whitelist regex | Epic 3: Vuln Remediation | `SASS-8` |

| **R-02** | **P1 (Critical)** | Insecure Direct Object Reference | Enforce `owner_id = session['user_id']` on all queries | Epic 3: Vuln Remediation | `SASS-9` |

| **R-03** | **P2 (High)** | Native Buffer Overflow | Bounded `memcpy` + input bounds validation + `-fstack-protector` | Epic 3: Vuln Remediation | `SASS-7` |

| **R-04** | **P2 (High)** | Cross-Site Request Forgery | Synchronizer token validation + `SameSite=Strict` cookies | Epic 3: Vuln Remediation | `SASS-10` |

| **R-05** | **P3 (Medium)** | Clickjacking | Enforce `X-Frame-Options: DENY` & CSP `frame-ancestors 'none'` | Epic 3: Vuln Remediation | `SASS-11` |

---

7. Residual Risk Statement

Following full implementation of the defensive controls:

- **Command Injection:** Residual risk is reduced to **Low (CVSS 0.0)** because direct shell invocation has been completely eliminated in favor of parameterized argument lists and strict alphanumeric regex matching.
- **IDOR:** Residual risk is reduced to **Low (CVSS 0.0)** because every database interaction enforces relational owner constraints at the SQL query level.
- **Buffer Overflow:** Residual risk is reduced to **Low (CVSS 0.0)** due to pre-copy length enforcement combined with hardware-assisted compiler stack canaries.
- **CSRF:** Residual risk is reduced to **Low (CVSS 0.0)** via per-session cryptographic synchronizer tokens and `SameSite` cookies.
- **Clickjacking:** Residual risk is reduced to **Low (CVSS 0.0)** via dual-layer HTTP header enforcement (`X-Frame-Options` and CSP `frame-ancestors`).
- **Operational Residual Risk:** Host OS physical compromise and compromised developer credentials remain the primary residual vectors, managed via repository branch protection, least privilege principles, and protected GitHub secrets.

Report Compiled & Verified by: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)